ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN A1

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH – CO5085

CNN vs TRANSFORMER TRÊN TẬP ẢNH LỚN

Đánh giá thực nghiệm CNN và Transformer dưới các chiến lược huấn luyện khác nhau

Giảng viên phụ trách: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Lớp – Nhóm: CH01 – DL4DL
Sinh viên thực hiện: Lê Tiến Đạt – 2310653
Nguyễn Lê Thảo Ly – 2312010

Thành phố Hồ Chí Minh, Tháng 10/2026

MỤC LỤC

1 GIỚI THIỆU **7**
1.1 Bối cảnh . . . 7
1.2 Mục tiêu và câu hỏi nghiên cứu . . . 7
1.3 Phạm vi . . . 8
1.4 Nguyên tắc so sánh . . . 8
1.5 Liên kết dự án . . . 8
1.6 Cấu trúc báo cáo . . . 9

2 PHÂN TÍCH TẬP DỮ LIỆU **10**
2.1 Food-101 . . . 10
2.2 Giấy phép và phạm vi sử dụng . . . 10
2.3 Chia dữ liệu . . . 11
2.4 Phân tích khám phá (EDA) . . . 11
2.5 Chất lượng nhãn . . . 12
2.6 Tiền xử lý, tăng cường dữ liệu và bộ nạp dữ liệu . . . 12

3 CÁC MÔ HÌNH CNN VÀ TRANSFORMER **14**
3.1 Kiến trúc CNN . . . 15
3.2 Kiến trúc Transformer . . . 15
3.3 Lý do chọn và inductive bias . . . 15
3.4 Các chế độ đóng băng . . . 16
3.5 Lưu ý khi diễn giải . . . 16

4 CHIẾN LƯỢC HUẤN LUYỆN **17**
4.1 Huấn luyện từ đầu (from-scratch) . . . 17
4.2 Sử dụng mô hình pretrained . . . 17
4.2.1 Đóng băng toàn bộ backbone (linear probing) . . . 17
4.2.2 Fine-tune một phần . . . 18
4.2.3 Fine-tune toàn bộ . . . 18

4.3 So sánh các chiến lược . . . 18
4.3.1 Tham số được cập nhật . . . 18
4.3.2 Chi phí tính toán (đo sơ bộ) . . . 18
4.3.3 Ưu điểm và hạn chế . . . 19

5 THIẾT KẾ THỰC NGHIỆM 20
5.1 Biến nghiên cứu . . . 20
5.2 Ma trận thí nghiệm . . . 20
5.3 Công thức huấn luyện chi tiết . . . 21
5.4 Quét learning rate . . . 23
5.5 Các thí nghiệm và ngân sách tính toán . . . 23
5.6 Giao thức đánh giá và phân tích . . . 24
5.7 Cách đo chi phí . . . 25

6 KẾT QUẢ THỰC NGHIỆM 26
6.1 Độ chính xác và F1 . . . 27
6.2 Diễn biến quá trình huấn luyện . . . 28
6.3 Ma trận nhầm lẫn . . . 28
6.4 Chi phí tính toán . . . 28
6.4.1 Chi phí huấn luyện . . . 29
6.4.2 Chi phí suy luận . . . 29
6.5 Ablation theo kích thước dữ liệu . . . 29
6.6 Kết quả phân tích bổ sung (ngoài yêu cầu tối thiểu) . . . 30

7 PHÂN TÍCH VÀ THẢO LUẬN 31
7.1 CNN và Transformer . . . 31
7.2 Vai trò của pretraining . . . 31
7.3 Ảnh hưởng của fine-tuning và freeze . . . 31
7.4 Phân tích lỗi . . . 32
7.4.1 Hiệu năng theo lớp . . . 32
7.4.2 Các mẫu dự đoán sai . . . 32
7.4.3 Kiểu lỗi của CNN và Transformer . . . 32
7.5 Hiệu chỉnh, ensemble, độ bền vững và diễn giải . . . 32
7.6 Đánh đổi giữa hiệu năng và chi phí . . . 32

8 TÁI LẬP THÍ NGHIỆM 33

9 KẾT LUẬN **35**
9.1 Tổng kết kết quả 35
9.2 Hạn chế và hướng phát triển 35

TÀI LIỆU THAM KHẢO **37**

A Ví dụ tệp cấu hình một lần chạy **39**

B Kết quả thực nghiệm bổ sung **41**

C Đo sơ bộ trên Colab (Tesla T4) **42**

DANH SÁCH BẢNG

2.1 Chia dữ liệu Food-101 của nhóm. 11

3.1 Các mô hình được so sánh (tham số tính với đầu ra 101 lớp). 14
3.2 Phần được huấn luyện ở mỗi chế độ và tỷ lệ tham số trainable (đo trên mô hình). 16

4.1 Chi phí tương đối của các chế độ so với full fine-tuning (Tesla T4; thời gian chỉ GPU và bộ nhớ đỉnh khi huấn luyện, batch 64). 19
4.2 Tóm tắt kỳ vọng về các chiến lược (sẽ được kiểm chứng ở Chương 7). 19

5.1 Biến độc lập, biến phụ thuộc và các yếu tố được giữ cố định. 20
5.2 Tám cấu hình lõi (cặp 1). 21
5.3 Công thức huấn luyện chính xác. 21
5.4 Lưới learning rate đỉnh cho mỗi mô hình. 23
5.5 Thí nghiệm theo thứ tự ưu tiên và chi phí ước tính. 24

6.1 Độ chính xác trên tập test (25 250 ảnh); mean ± std trên 3 seed với cấu hình pretrained, 1 seed với from-scratch. 27
6.2 Các cặp lớp bị nhầm nhiều nhất trên test (full fine-tuning). 28
6.3 Chi phí huấn luyện. Cột "chỉ GPU" dùng để so sánh; cột "Colab" là thời gian thực trong lần huấn luyện thật và bị giới hạn bởi bộ nạp dữ liệu (xem Mục 5.7). 29
6.4 Chi phí suy luận (chỉ phụ thuộc kiến trúc; AMP fp16; Tesla T4, PyTorch 2.11.0+cu130, timm 1.0.30, đo bằng `cost_bench.py`). Latency batch 1 bị chi phối bởi chi phí khởi chạy kernel nên không tỷ lệ với GMACs. Hai lần đo trên hai phiên lệch nhau tới khoảng 7%; đây là lần đo thứ hai. 29

8.1 Quy trình chuẩn và cách nhóm thực sự chạy. 33

C.1 Thông lượng bộ nạp dữ liệu (img/s) trên ảnh đã resize 256 px, batch 64. "Float" chuẩn hóa trên CPU; "uint8" chuẩn hóa trên GPU. 42
C.2 Tốc độ huấn luyện chỉ GPU (batch tổng hợp trên GPU) và bộ nhớ GPU đỉnh. . . 43

C.3 Tốc độ huấn luyện end-to-end (bộ nạp uint8 + RandAugment, 2 worker, Mixup/CutMix) và tỷ lệ thời gian chờ dữ liệu. 43

DANH SÁCH HÌNH VẼ

2.1 Mẫu ảnh Food-101 và ảnh sau augmentation (sẽ bổ sung). 12

6.1 So sánh độ chính xác và tốc độ thích ứng giữa các chiến lược huấn luyện. 28

Chương 1

GIỚI THIỆU

1.1 Bối cảnh

Mạng tích chập (CNN) và Vision Transformer (ViT) là hai họ kiến trúc chủ đạo trong thị giác máy tính. CNN mang sẵn *inductive bias* về tính cục bộ và bất biến tịnh tiến; ViT dựa trên chú ý toàn cục và thường cần nhiều dữ liệu hoặc trọng số pretrained để đạt hiệu năng cao [3, 14]. Khi dữ liệu và tài nguyên tính toán hạn chế, câu hỏi thực tế là nên chọn họ kiến trúc nào, nên khởi tạo từ đâu, và nên cập nhật bao nhiêu phần của mô hình.

1.2 Mục tiêu và câu hỏi nghiên cứu

Bài tập lớn A1 so sánh CNN và Transformer trên một tập ảnh lớn do nhóm chọn, dưới nhiều chế độ huấn luyện: huấn luyện từ đầu (*from-scratch*) so với dùng trọng số pretrained, và ba kiểu đóng băng khi dùng pretrained (đóng băng toàn bộ backbone, đóng băng một phần, fine-tune toàn bộ). Nhóm tìm câu trả lời cho bốn câu hỏi:

RQ1. *Khi nào chiến lược freeze có lợi?* Đánh đổi giữa chi phí tính toán, nguy cơ overfitting và độ chính xác.

RQ2. *Khi nào ViT cần pretrain?* Khoảng cách giữa from-scratch và pretrained thay đổi thế nào theo kích thước dữ liệu huấn luyện, và so với CNN ra sao.

RQ3. *Chi phí tài nguyên:* số tham số trainable, GMACs, thời gian huấn luyện, thông lượng và độ trễ suy luận, bộ nhớ GPU đỉnh.

RQ4. *Phân tích lỗi và độ tin cậy:* nhầm lẫn giữa các lớp, khác biệt lỗi giữa CNN và ViT, hiệu chỉnh độ tin cậy (calibration), tính bổ trợ khi ensemble và độ bền vững trước nhiễu.

1.3 Phạm vi

Nhóm chọn tập **Food-101** [1] (101 lớp món ăn, khoảng 100 nghìn ảnh) và so sánh ResNet-50 [7, 16] với DeiT-Small [14] (cặp 1, phần cam kết). Cặp ConvNeXt-Tiny [10] và Swin-Tiny [9] (cặp 2) là phần mở rộng, chỉ chạy khi còn tài nguyên tính toán. Các thí nghiệm gồm tám cấu hình lõi, ablation theo kích thước dữ liệu, và lặp lại ba seed cho các cấu hình pretrained. Sau đó là các phân tích chỉ cần suy luận (hiệu chỉnh, ensemble, độ bền vững, diễn giải) được trình bày như phần mở rộng ngoài yêu cầu tối thiểu của đề bài. Báo cáo cũng trình bày quy trình tái lập, tách bạch giữa *quy trình chuẩn* (chỉ cần mã nguồn và báo cáo) và *cách nhóm thực sự chạy* trên Google Colab miễn phí.

1.4 Nguyên tắc so sánh

Để chênh lệch kết quả quy được cho kiến trúc và chế độ huấn luyện, nhóm giữ cố định các yếu tố sau cho mọi mô hình:

- cùng split dữ liệu (cố định bằng `split.json`) và cùng phép biến đổi khi đánh giá;
- cùng một công thức huấn luyện (tối ưu hóa, lịch learning rate, augmentation, regularization), chỉ khác learning rate đỉnh được chọn bằng cùng một quy trình quét trên validation;
- cùng số epoch theo chế độ (10 epoch cho pretrained, 40 epoch cho from-scratch) và báo cáo epoch cuối, không chọn checkpoint tốt nhất;
- hai mô hình trong mỗi cặp có số tham số và GMACs xấp xỉ nhau (Bảng 3.1);
- cùng một công cụ, quy ước và GPU để đo chi phí (Mục 5.7).

Những điểm không thể đồng nhất, như công thức pretrain khác nhau của từng bộ trọng số, được nêu ở Mục 9.2.

1.5 Liên kết dự án

Các tài nguyên liên quan đến bài tập A1:

- Kho mã nguồn GitHub (thư mục `A1`): `https://github.com/broistg/deep-learning-CO5085/tree/main/A1`
- GitHub Page của A1: `https://broistg.github.io/deep-learning-CO5085/assignments/a1.html`

- Landing page của nhóm: `https://broistg.github.io/deep-learning-CO5085/index.html`
- Video demo: *[sẽ cập nhật]*
- Trọng số và logits (GitHub Release): *[sẽ cập nhật]*

1.6 Cấu trúc báo cáo

Chương 2 mô tả tập dữ liệu, tiền xử lý và bộ nạp dữ liệu; Chương 3 giới thiệu kiến trúc các mô hình; Chương 4 trình bày các chiến lược huấn luyện; Chương 5 nêu thiết kế thực nghiệm; Chương 6 trình bày kết quả; Chương 7 phân tích và thảo luận; Chương 8 trình bày cách tái lập; Chương 9 kết luận và nêu hạn chế.

Trạng thái tài liệu. Các Chương 2–5 và Chương 8 tương ứng mốc A1.1 (dataset và protocol). Các mục đánh dấu *[sẽ cập nhật]* sẽ được hoàn thiện ở mốc A1.2.

Chương 2

PHÂN TÍCH TẬP DỮ LIỆU

2.1 Food-101

Food-101 [1] gồm 101 lớp món ăn, mỗi lớp 1 000 ảnh: 750 ảnh huấn luyện và 250 ảnh kiểm thử, tổng cộng 101 000 ảnh. Ảnh được thu thập từ Foodspotting; theo tác giả, ảnh huấn luyện cố ý không được làm sạch nên còn chứa nhiễu (kể cả nhiễu nhãn), còn ảnh kiểm thử được rà soát nhãn thủ công. Trang dữ liệu: `https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/`; bản trên Hugging Face: `https://huggingface.co/datasets/ethz/food101`.

Lý do chọn.

- 101 lớp phân loại mịn với khoảng 75 nghìn ảnh huấn luyện: lớn và khó hơn các tập đã dùng ở bài E1, đủ để khác biệt giữa CNN và ViT, và tác dụng của pretrain/freeze, có ý nghĩa.
- Các lớp cân bằng tuyệt đối nên không cần reweighting, oversampling hay focal loss.
- Cạnh dài tối đa 512 px, phù hợp đầu vào 224×224 và chi phí tải dữ liệu thấp trên Colab.
- Ảnh được thu thập từ Foodspotting, không phải từ ImageNet. Tuy nhiên nhóm *chưa kiểm chứng độc lập* mức độ trùng lặp ở cấp ảnh giữa Food-101 và dữ liệu pretrain (ImageNet-1k), nên không khẳng định rằng các mô hình pretrained chưa từng thấy ảnh kiểm thử. ImageNet-1k có một số lớp liên quan đến món ăn, vì vậy lợi thế của pretrain có thể một phần đến từ sự gần gũi về miền dữ liệu (xem Mục 9.2).

2.2 Giấy phép và phạm vi sử dụng

Giấy phép của Food-101 là **chưa rõ**: dataset card trên Hugging Face ghi license là "unknown" [4]. Phần Licensing Information của thẻ này nêu rằng ảnh đến từ Foodspotting, không thuộc sở hữu của ETH Zürich, và mọi việc sử dụng vượt quá "scientific fair use" phải được thỏa thuận với chủ sở

hữu ảnh theo điều khoản của Foodspotting [4]. Theo bản lưu trữ ngày 23/12/2017 của điều khoản Foodspotting (hiệu lực từ 17/10/2012) [5], người đóng góp giữ bản quyền nội dung, Foodspotting được cấp quyền sử dụng rộng (gồm quyền cấp phép lại), còn việc bên thứ ba sử dụng nội dung cần văn bản cho phép của Foodspotting hoặc chủ sở hữu.

Vì nhóm chưa kiểm chứng độc lập phạm vi giấy phép áp dụng cho bản phân phối Food-101 và cho cách sử dụng của nhóm, tập dữ liệu **không** được coi là tài nguyên mở cho việc sử dụng và phân phối không hạn chế. Nhóm chỉ dùng dữ liệu cho mục đích học tập và nghiên cứu trong bài tập lớn của môn học, và **không công bố** ảnh gốc, ảnh đã resize, ảnh đã tăng cường hoặc làm nhiễu, hay bất kỳ cache nào chứa ảnh; các bản này được sinh lại bằng mã nguồn với seed cố định. Nhóm công bố mã nguồn, cấu hình, trọng số mô hình và logits dự đoán (không chứa ảnh). Báo cáo chỉ chèn một số ít ảnh minh họa cỡ nhỏ khi cần trình bày kết quả phân tích. Mọi hình thức phân phối công khai khác chỉ được xem xét sau khi làm rõ điều kiện cấp phép. Nguồn dữ liệu: [1].

2.3 Chia dữ liệu

Tập train chính thức (750 ảnh mỗi lớp) được chia phân tầng thành 700 ảnh huấn luyện và 50 ảnh validation mỗi lớp bằng seed 42; tập test chính thức được giữ nguyên và không dùng để huấn luyện hay chọn siêu tham số. Danh sách ảnh được lưu trong `split.json` (SHA-256, 16 ký tự đầu: `75ed81bfe380c089`) và được commit cùng mã nguồn; chương trình kiểm tra giá trị này khi khởi động.

Bảng 2.1: Chia dữ liệu Food-101 của nhóm.

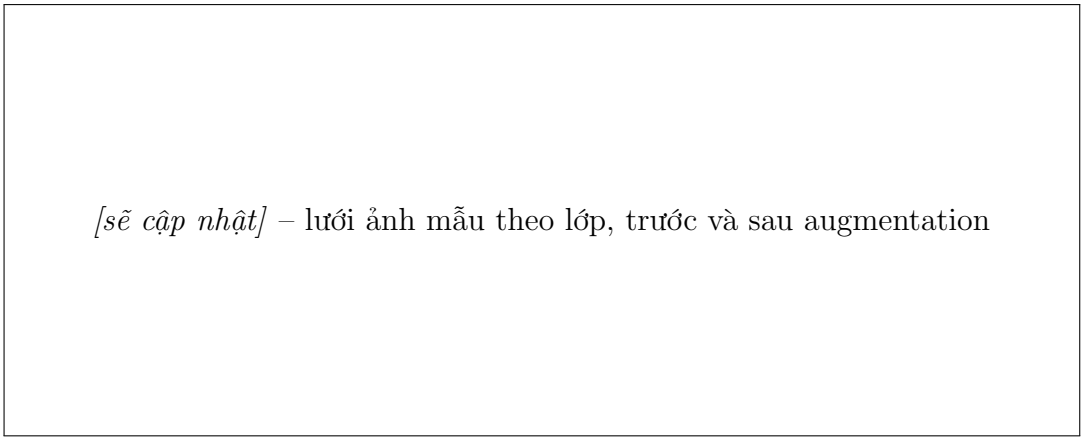
Tập	Số ảnh	Ảnh/lớp	Cách tạo
Huấn luyện	70 700	700	Phần còn lại của tập train chính thức
Validation	5 050	50	Lấy phân tầng từ tập train chính thức, seed 42
Kiểm thử	25 250	250	Tập test chính thức, giữ nguyên

Validation chỉ dùng để chọn learning rate và để fit temperature khi hiệu chỉnh độ tin cậy. Do validation lấy từ phần train có nhiễu nhãn, nó chỉ được dùng cho các mục đích này; mọi số liệu báo cáo cuối cùng là trên tập test, ở epoch cuối của mỗi lần chạy (không chọn checkpoint tốt nhất theo validation).

2.4 Phân tích khám phá (EDA)

Trên một mẫu ngẫu nhiên 200 ảnh huấn luyện, cạnh ngắn có trung vị 512 px, cạnh dài lớn nhất là 512 px, và toàn bộ ảnh đều ở chế độ RGB; không có ảnh nào không đọc được. Kiểm tra

tự động xác nhận 101 lớp, các tập rời nhau, mỗi lớp có đúng 700/50/250 ảnh, và kích thước chính thức 75 750 (train, trước khi tách validation) và 25 250 (test).



Hình 2.1: Mẫu ảnh Food-101 và ảnh sau augmentation (sẽ bổ sung).

2.5 Chất lượng nhãn

Nhãn huấn luyện được gán tự động theo phân loại món của Foodspotting và không được rà soát thủ công [1], vì vậy độ chính xác tối đa đạt được bị giới hạn bởi nhiễu nhãn. Ở phần phân tích lỗi, nhóm minh họa hiện tượng này bằng các ảnh huấn luyện có loss cao nhất (*[sẽ cập nhật]*).

2.6 Tiền xử lý, tăng cường dữ liệu và bộ nạp dữ liệu

Mục này mô tả quy trình dữ liệu ở mức khái niệm; giá trị chính xác của từng tham số nằm ở Bảng 5.3 (Chương 5) và trong các tệp cấu hình.

Tiền xử lý offline. Mọi ảnh được resize một lần về cạnh ngắn 256 px (Pillow, nội suy bicubic, JPEG chất lượng 90; chỉ thu nhỏ, không phóng to) để giảm chi phí giải mã khi huấn luyện. Bước này nằm trong mã nguồn của repo (Chương 8); bản cache chỉ là cách tiết kiệm thời gian chạy lại bước đó, không đổi kết quả của bước.

Tăng cường dữ liệu khi huấn luyện. Mỗi ảnh được cắt ngẫu nhiên theo tỷ lệ diện tích 0,25–1 và tỷ lệ khung hình 3/4–4/3 rồi đưa về 224×224, lật ngang với xác suất 0,5, và áp dụng RandAugment [2] (hai phép biến đổi ngẫu nhiên, độ mạnh cố định ở mức nhẹ). Sau đó, mỗi batch được trộn bằng Mixup [18] hoặc CutMix [17] (chọn một trong hai) cùng label smoothing. Mục đích là giảm overfitting trên tập huấn luyện 70 700 ảnh có nhiễu nhãn, đặc biệt với các chế độ huấn luyện nhiều tham số. Cùng một công thức được dùng cho cả hai họ kiến trúc.

Khi đánh giá. Ảnh được resize về cạnh ngắn 256, cắt giữa 224×224 và chuẩn hóa mean/std ImageNet, giống nhau cho cả bốn mô hình; không có augmentation.

Bộ nạp dữ liệu. Batch 64, xáo trộn mỗi epoch, hai worker (Colab miễn phí chỉ có 2 nhân CPU). Để bộ nạp không là nút cổ chai, các phép biến đổi trên CPU chỉ cho ra tensor kiểu uint8 và việc đổi sang số thực cùng chuẩn hóa mean/std được chuyển sang GPU; cách này tăng thông lượng bộ nạp từ khoảng 279 lên khoảng 372 ảnh/giây khi có RandAugment (Phụ lục C). Dù vậy, với các chế độ freeze (GPU xử lý nhanh) bộ nạp vẫn là nút cổ chai, nên thời gian thực tế trên Colab gần như không phản ánh được lợi ích tính toán của freeze; vì vậy chi phí được so sánh bằng phép đo kiểm soát chỉ trên GPU (Mục 5.7).

Chương 3

CÁC MÔ HÌNH CNN VÀ TRANSFORMER

Hai họ kiến trúc được so sánh ở cùng quy mô tham số và chi phí tính toán, đều dùng trọng số pretrained ImageNet-1k từ thư viện `timm` [15], tức cùng dữ liệu pretrain. Cặp 1 là phần cam kết; cặp 2 là phần mở rộng, chạy nếu còn tài nguyên tính toán.

Bảng 3.1: Các mô hình được so sánh (tham số tính với đầu ra 101 lớp).

Họ	Mô hình	Trọng số (timm)	Tham số (M)	GMACs
CNN (cặp 1)	ResNet-50 [7]	`resnet50.a1_in1k` [16]	23,71	4,087
Transformer (cặp 1)	DeiT-Small/16 [14]	`deit_small_patch16_224.fb_in1k`	21,70	4,599
CNN (cặp 2)	ConvNeXt-Tiny [10]	`convnext_tiny.fb_in1k`	27,90	4,455
Transformer (cặp 2)	Swin-Tiny [9]	`swin_tiny_patch4_window7_224.ms_in1k`	27,60	4,490

Quy ước đo chi phí tính toán. GMACs được đo bằng *một* công cụ và *một* quy ước cho cả bốn kiến trúc: `torch.utils.flop_counter.FlopCounterMode`, một lượt forward với đầu vào $1 \times 3 \times 224 \times 224$ và đầu ra 101 lớp, tắt fused attention để các phép nhân ma trận của attention được đếm; chỉ đếm phép nhân ma trận và tích chập (không đếm phép element-wise, softmax, chuẩn hóa). Vì `FlopCounterMode` tính một phép nhân-cộng (MAC) bằng hai FLOPs, GMACs = GFLOPs/2. Nhóm báo cáo GMACs của `FlopCounterMode` và đối chiếu bằng `fvcore`: bốn mô hình cho 4,111, 4,608, 4,470, 4,509 GMACs (ResNet-50, DeiT-S, ConvNeXt-T, Swin-T), cao hơn 0,2–0,6% so với `FlopCounterMode`; chênh lệch nhỏ này có thể do fvcore đếm thêm một

số phép chuẩn hóa và pooling. Các con số "FLOPs" trong các bài báo gốc thực chất là MACs và khớp với kết quả đo trong vòng 1%.

3.1 Kiến trúc CNN

ResNet-50 [7]. Một lớp tích chập 7×7 bước 2 cùng max-pooling, tiếp theo là bốn stage gồm 3, 4, 6 và 3 khối bottleneck (1×1, 3×3, 1×1) với kết nối tắt cộng đầu vào vào đầu ra, rồi global average pooling và một lớp tuyến tính. Chuẩn hóa dùng BatchNorm. Kết nối tắt giúp gradient lan truyền qua mạng sâu; phần lớn tham số nằm ở stage cuối (`layer4`). Trọng số `a1_in1k` đến từ công thức huấn luyện "ResNet strikes back" [16].

ConvNeXt-Tiny [10] (cặp 2). Một CNN được thiết kế lại theo các thực hành của Transformer: lớp "patchify" 4×4 bước 4, bốn stage gồm 3, 3, 9 và 3 khối, mỗi khối dùng tích chập depthwise 7×7, LayerNorm, khối MLP dạng inverted bottleneck và kích hoạt GELU. Mục đích của việc đưa vào là tách ảnh hưởng của họ kiến trúc khỏi ảnh hưởng của đời thiết kế.

3.2 Kiến trúc Transformer

DeiT-Small [3, 14]. Ảnh 224×224 được cắt thành 196 patch 16×16, mỗi patch được chiếu tuyến tính thành vector 384 chiều, thêm token CLS và embedding vị trí. Chuỗi 197 token đi qua 12 khối Transformer (6 đầu chú ý, LayerNorm, MLP); đầu ra của token CLS đi vào lớp phân loại. Chú ý toàn cục có từ khối đầu tiên và chi phí tính chú ý tăng theo bình phương số token. Trọng số `fb_in1k` được huấn luyện chỉ trên ImageNet-1k với augmentation và regularization mạnh.

Swin-Tiny [9] (cặp 2). Transformer phân cấp: patch 4×4, bốn stage gồm 2, 2, 6 và 2 khối, chú ý được tính trong cửa sổ cục bộ 7×7 và cửa sổ được dịch xen kẽ giữa các khối để thông tin lan sang cửa sổ lân cận; giữa các stage có bước gộp patch làm giảm độ phân giải như CNN. Swin vì thế đưa lại tính cục bộ và cấu trúc phân cấp vào Transformer.

3.3 Lý do chọn và inductive bias

Cặp 1 (ResNet-50 và DeiT-S) có số tham số xấp xỉ nhau (Bảng 3.1) và đều là mô hình "tiêu chuẩn" của mỗi họ, nên là phép so sánh sạch nhất giữa tích chập và chú ý toàn cục. Cặp 2 thêm hai mô hình hiện đại hơn có cấu trúc gần nhau (phân cấp bốn stage), để xem kết luận có phụ thuộc vào đời thiết kế hay không. Kỳ vọng ban đầu, sẽ được kiểm chứng bằng số liệu ở Chương 7, là CNN nhờ tính cục bộ và chia sẻ trọng số cần ít dữ liệu hơn để học tốt từ đầu, còn ViT ít giả định hơn về cấu trúc ảnh nên phụ thuộc nhiều hơn vào dữ liệu hoặc pretrain.

3.4 Các chế độ đóng băng

Bảng 3.2 cho phần được huấn luyện của từng chế độ; tỷ lệ tham số trainable được đo trên mô hình thực (đầu ra 101 lớp). Với các kiến trúc phân cấp (ResNet, ConvNeXt, Swin), freeze một phần nghĩa là chỉ huấn luyện stage cuối cùng cùng phần đầu ra; với DeiT là nửa sau của các block cùng chuẩn hóa cuối và phần đầu ra. Các tỷ lệ này khác nhau giữa các kiến trúc nên được ghi rõ ở mọi bảng kết quả.

Bảng 3.2: Phần được huấn luyện ở mỗi chế độ và tỷ lệ tham số trainable (đo trên mô hình).

Chế độ	Phần được huấn luyện	ResNet-50	DeiT-S	ConvNeXt-T	Swin-T
Freeze toàn bộ backbone	Chỉ lớp phân loại	0,9%	0,2%	0,3%	0,3%
Freeze một phần	Stage cuối + đầu ra; DeiT: block 7–12 + norm + đầu ra	64,0%	49,2%	55,7%	56,0%
Full fine-tuning	Toàn bộ	100%	100%	100%	100%
From-scratch	Toàn bộ, khởi tạo ngẫu nhiên	100%	100%	100%	100%

Phần bị đóng băng được đặt ở chế độ eval, nên BatchNorm không cập nhật thống kê.

3.5 Lưu ý khi diễn giải

Không dùng ViT-Base/16 (khoảng 86M tham số) vì so sánh với ResNet-50 sẽ lẫn lộn ảnh hưởng của kiến trúc với ảnh hưởng của quy mô mô hình, và chi phí huấn luyện cao hơn nhiều. Các trọng số pretrained của hai họ được huấn luyện bằng các công thức khác nhau (ví dụ ResNet-50 "a1" dùng công thức huấn luyện mạnh [16]), nên một phần khác biệt giữa hai họ có thể đến từ công thức pretrain chứ không chỉ từ kiến trúc. Cặp ConvNeXt-Tiny/Swin-Tiny được đưa vào để kiểm tra khác biệt do họ kiến trúc hay do đời thiết kế.

Chương 4

CHIẾN LƯỢC HUẤN LUYỆN

Chương này mô tả các chiến lược huấn luyện được so sánh và những gì chúng khác nhau về tham số được cập nhật và chi phí. Các nhận định về hiệu quả là *kỳ vọng* dựa trên kiến thức chung, được kiểm chứng bằng số liệu ở Chương 7; chỉ các số đo chi phí (đo sơ bộ trên Tesla T4) là kết quả thực.

4.1 Huấn luyện từ đầu (from-scratch)

Mô hình được khởi tạo ngẫu nhiên và mọi tham số được học chỉ từ tập huấn luyện của nhóm (70 700 ảnh). Mô hình phải học cả các đặc trưng cấp thấp (cạnh, kết cấu) lẫn đặc trưng cấp cao của món ăn từ lượng dữ liệu nhỏ hơn nhiều so với ImageNet-1k (khoảng 1,28 triệu ảnh), nên thường cần nhiều epoch hơn, augmentation mạnh hơn và có nguy cơ overfitting cao hơn. Kỳ vọng là kiến trúc có inductive bias mạnh (CNN) học tốt hơn từ ít dữ liệu so với kiến trúc ít giả định (ViT); nhóm kiểm chứng bằng from-scratch 40 epoch cho cả hai họ và ablation theo kích thước dữ liệu. Đây cũng là chế độ đắt nhất: 40 epoch cho mỗi mô hình so với 10 epoch ở các chế độ pretrained.

4.2 Sử dụng mô hình pretrained

Transfer learning khởi tạo mô hình từ trọng số học ở tác vụ nguồn (ImageNet-1k) và điều chỉnh cho tác vụ đích (Food-101, 101 lớp). Ba chiến lược sau khác nhau ở phần trọng số được cập nhật (Bảng 3.2).

4.2.1 Đóng băng toàn bộ backbone (linear probing)

Mọi trọng số của backbone giữ nguyên, chỉ lớp phân loại 101 lớp được huấn luyện từ khởi tạo ngẫu nhiên; phần đóng băng được đặt ở chế độ eval nên BatchNorm không cập nhật thống kê.

Chiến lược này ít tham số cần học nhất (dưới 1%), ít nguy cơ overfitting và rẻ nhất về bộ nhớ, nhưng chất lượng bị giới hạn bởi mức độ phù hợp của đặc trưng ImageNet với tập đích.

4.2.2 Fine-tune một phần

Các tầng đầu (đặc trưng tổng quát) được đóng băng; stage cuối cùng của CNN hoặc nửa sau các khối của DeiT, cùng lớp phân loại, được huấn luyện. Các tầng sau thường mang đặc trưng riêng cho tác vụ nguồn hơn nên là phần có lợi nhất khi điều chỉnh, còn tầng đầu giữ nguyên giúp giảm tham số cần cập nhật và nguy cơ overfitting. Vị trí cắt là lựa chọn thiết kế (một mức duy nhất cho mỗi kiến trúc) và không được tối ưu hóa.

4.2.3 Fine-tune toàn bộ

Khởi tạo từ trọng số pretrained rồi cập nhật mọi tham số, với learning rate đỉnh nhỏ để không phá đặc trưng đã học. Đây là chiến lược linh hoạt nhất và thường cho độ chính xác cao nhất khi có đủ dữ liệu, nhưng tốn bộ nhớ nhất và có nguy cơ overfitting hoặc quên bớt đặc trưng tổng quát khi tập đích nhỏ hoặc nhiễu.

4.3 So sánh các chiến lược

4.3.1 Tham số được cập nhật

Tỷ lệ tham số trainable của từng chế độ và kiến trúc nằm ở Bảng 3.2: dưới 1% khi freeze toàn bộ, khoảng một nửa đến hai phần ba khi freeze một phần, và 100% khi fine-tune toàn bộ hoặc huấn luyện từ đầu. Do cách cắt khác nhau, cùng một chế độ "freeze một phần" không ứng với cùng tỷ lệ giữa các kiến trúc (64,0% ở ResNet-50 so với 49,2% ở DeiT-S), nên khi so sánh chế độ này giữa hai họ cần đọc kèm tỷ lệ.

4.3.2 Chi phí tính toán (đo sơ bộ)

Giảm số tham số trainable không giảm tỷ lệ thuận chi phí, vì lượt forward qua backbone vẫn cần cho mọi bước; freeze chỉ bỏ phần backward và trạng thái optimizer của phần bị đóng băng, và cho phép không lưu activation của nó. Bảng 4.1 cho thấy hệ quả đo được trên Tesla T4 (batch 64, AMP fp16, batch tổng hợp trên GPU; chi tiết ở Mục 5.7).

Bảng 4.1: Chi phí tương đối của các chế độ so với full fine-tuning (Tesla T4; thời gian chỉ GPU và bộ nhớ đỉnh khi huấn luyện, batch 64).

Kiến trúc	Chế độ	s/epoch	Nhanh hơn full	Bộ nhớ (MB)	Bộ nhớ so với full
ResNet-50	Freeze toàn bộ	70,8	2,9×	517	16%
ResNet-50	Freeze một phần	87,6	2,3×	692	22%
ResNet-50	Full	205,0	1,0×	3 206	100%
DeiT-S	Freeze toàn bộ	60,0	3,3×	333	13%
DeiT-S	Freeze một phần	126,1	1,6×	1 383	54%
DeiT-S	Full	197,3	1,0×	2 550	100%

Kết quả cho thấy lợi ích của freeze về bộ nhớ lớn hơn lợi ích về thời gian, và lợi ích phụ thuộc kiến trúc: freeze một phần ở DeiT-S (nửa sau các khối) tiết kiệm ít hơn ở ResNet-50 (chỉ stage cuối). Ngoài ra, trên Colab miễn phí, nút cổ chai ở bộ nạp dữ liệu làm các chế độ freeze gần như không nhanh hơn về thời gian thực (Mục 5.7), nên lợi ích về thời gian chỉ thấy rõ khi dữ liệu được cấp đủ nhanh.

4.3.3 Ưu điểm và hạn chế

Bảng 4.2: Tóm tắt kỳ vọng về các chiến lược (sẽ được kiểm chứng ở Chương 7).

Chiến lược	Ưu điểm kỳ vọng	Hạn chế kỳ vọng
From-scratch	Không phụ thuộc miền dữ liệu của tác vụ nguồn; kiến trúc tự do	Đắt nhất; cần nhiều dữ liệu và epoch; ViT thường kém hơn CNN khi dữ liệu ít
Freeze toàn bộ	Rẻ nhất; ít overfitting; huấn luyện nhanh hội tụ	Giới hạn bởi đặc trưng ImageNet; thường kém nhất trong các chế độ pretrained
Freeze một phần	Cân bằng giữa chi phí và khả năng thích ứng	Vị trí cắt là lựa chọn cố định; lợi ích phụ thuộc kiến trúc
Fine-tune toàn bộ	Thường cho độ chính xác cao nhất	Tốn bộ nhớ nhất; nhạy với learning rate; có thể quên đặc trưng tổng quát

Chương 5

THIẾT KẾ THỰC NGHIỆM

5.1 Biến nghiên cứu

Bảng 5.1: Biến độc lập, biến phụ thuộc và các yếu tố được giữ cố định.

Loại biến	Nội dung
Độc lập	Họ kiến trúc (CNN / Transformer); khởi tạo (ngẫu nhiên / pretrained ImageNet-1k); chế độ huấn luyện (freeze toàn bộ / freeze một phần / full fine-tuning); tỷ lệ dữ liệu huấn luyện (10, 25, 50, 100%) trong ablation; seed
Phụ thuộc	Top-1, top-5, macro-F1 trên test; số tham số trainable; GMACs; thời gian huấn luyện; thông lượng và độ trễ suy luận; bộ nhớ GPU đỉnh; ECE, NLL, độ bền vững
Giữ cố định	Split dữ liệu; công thức augmentation và regularization; bộ tối ưu; hình dạng lịch learning rate; batch; số epoch theo chế độ; phép biến đổi khi đánh giá; quy ước đo chi phí (Mục 1.4)

5.2 Ma trận thí nghiệm

Với mỗi mô hình ở cặp 1, nhóm chạy bốn cấu hình trên cùng một split, tổng cộng tám cấu hình lõi (Bảng 5.2).

Bảng 5.2: Tám cấu hình lõi (cặp 1).

Mô hình	Chế độ	Khởi tạo	Phần huấn luyện	Epoch
ResNet-50, DeiT-S	From-scratch	Ngẫu nhiên	Toàn bộ	40
ResNet-50, DeiT-S	Freeze toàn bộ backbone	ImageNet-1k	Lớp phân loại	10
ResNet-50, DeiT-S	Freeze một phần	ImageNet-1k	Xem Bảng 3.2	10
ResNet-50, DeiT-S	Full fine-tuning	ImageNet-1k	Toàn bộ	10

5.3 Công thức huấn luyện chi tiết

Một công thức duy nhất được dùng cho mọi mô hình và chế độ; các giá trị này nằm trong các tệp cấu hình YAML được commit cùng mã nguồn (Phụ lục A).

Bảng 5.3: Công thức huấn luyện chính xác.

Hạng mục	Giá trị chính xác
Phần mềm	Python 3.13, PyTorch 2.11.0, torchvision 0.26.0, timm 1.0.30; phiên bản Pillow và NumPy được ghi vào `results/*.json` của mỗi lần chạy.
Resize offline	Pillow: cạnh ngắn → 256 px (chỉ thu nhỏ), nội suy bicubic, JPEG chất lượng 90.
Biến đổi khi train	(1) `RandomResizedCrop(224,scale=(0.25,1.0),ratio=(3/4,4/3))`, nội suy bilinear (`scale` khác mặc định $(0{,}08, 1)$); (2) `RandomHorizontalFlip(p=0.5)`; (3) `RandAugment(num_ops=2,magnitude=9,num_magnitude_bins=31,interpolation=NEAREST,fill=None)` của torchvision [2]; (4) `PILToTensor` (uint8); (5) trên GPU: chuẩn hóa mean $(0{,}485; 0{,}456; 0{,}406)$, std $(0{,}229; 0{,}224; 0{,}225)$; (6) Mixup/CutMix.
Độ mạnh RandAugment	Mỗi ảnh nhận 2 phép chọn ngẫu nhiên có hoàn lại trong 14 phép, luôn được áp dụng, dấu của phép có dấu chọn ngẫu nhiên. Chỉ số `magnitude=9` trên 31 mức tương ứng $9/30 = 30\%$ biên độ tối đa của mỗi phép (ví dụ xoay $\pm 9°$, shear $\pm 0{,}09$, độ sáng/màu/tương phản $\pm 0{,}27$). Mức này *nhẹ hơn nhiều* so với `rand-m9` trong timm/DeiT ($9/10 = 90\%$ biên độ). Cấu hình được giữ cố định cho mọi mô hình và không tinh chỉnh.

Hạng mục	Giá trị chính xác
Mixup/CutMix	`timm.data.Mixup(mixup_alpha=0.8,cutmix_alpha=1.0,cutmix_minmax=None,prob=1.0,switch_prob=0.5,mode='batch',correct_lam=True,label_smoothing=0.1,num_classes=101)` [17, 18]. Mỗi batch (`prob=1.0`) nhận *đúng một* trong hai: Mixup với xác suất 0,5 ($\lambda \sim \text{Beta}(0{,}8; 0{,}8)$) hoặc CutMix với xác suất 0,5 ($\lambda \sim \text{Beta}(1; 1)$, sau đó λ được hiệu chỉnh theo diện tích hộp thực tế sau khi cắt biên); hai phương pháp loại trừ nhau trong một batch. Mỗi mẫu được ghép với mẫu đối xứng trong batch (đảo thứ tự batch); batch 64 (chẵn), `drop_last=True`. Việc chọn phương pháp và λ dùng bộ sinh số ngẫu nhiên toàn cục của NumPy trong tiến trình chính, nên được seed và được lưu/khôi phục cùng checkpoint.
Nhãn mềm và loss	Nhãn đích $y = \lambda\, y_s(a) + (1-\lambda)\, y_s(b)$ với $y_s(c) = (1-\varepsilon)\,\text{onehot}(c) + \varepsilon/101$, $\varepsilon = 0{,}1$ [13]. Label smoothing được áp dụng **một lần duy nhất, bên trong Mixup**. Loss huấn luyện là soft-target cross-entropy $-\sum_k y_k \log \text{softmax}(z)_k$ tính ở fp32 và **không** truyền thêm tham số label smoothing vào hàm loss. Khi đánh giá (validation/test) dùng cross-entropy với nhãn cứng, không smoothing; NLL và ECE tính trên nhãn cứng.
Biến đổi khi đánh giá	`Resize(256)` (bilinear) → `CenterCrop(224)` → chuẩn hóa mean/std ImageNet; cùng một phép cho cả bốn mô hình.
Tối ưu hóa	AdamW [11] ($\beta = (0{,}9; 0{,}999)$, $\epsilon = 10^{-8}$), weight decay 0,05 chỉ cho tham số từ 2 chiều trở lên (không áp dụng cho bias và tham số chuẩn hóa); không gradient clipping; không EMA; `drop_rate=0` và `drop_path_rate=0` được đặt tường minh cho mọi mô hình (không dùng giá trị mặc định riêng của từng mô hình). Mixed precision fp16 với GradScaler động.
Lịch learning rate	Theo từng iteration: warmup tuyến tính từ 1% LR đỉnh lên LR đỉnh trong 5% tổng số bước đầu (0,5 epoch với 10 epoch; 2 epoch với 40 epoch), sau đó giảm cosine tới 1% LR đỉnh ở bước cuối. Cùng hình dạng lịch cho lượt quét LR và các lần chạy chính.
Batch và dữ liệu	Batch 64, `drop_last=True`, 2 worker, xáo trộn mỗi epoch. Thứ tự dữ liệu và ngẫu nhiên của augmentation sinh từ generator seed theo (seed, epoch).
Seed và tái lập	Seed 0/1/2 (lượt quét LR dùng seed 0). Trạng thái RNG của Python, NumPy, PyTorch, CUDA cùng GradScaler, optimizer, scheduler được lưu trong checkpoint mỗi epoch; khôi phục RNG cho kết quả trùng khớp từng bit trên một mô hình thử khi bật chế độ xác định. Với huấn luyện thật, `cudnn.benchmark` được bật để nhanh hơn nên kết quả không bảo đảm trùng từng bit giữa các lần chạy hoặc giữa các loại GPU; sai khác được ước lượng từ 3 seed.

Epoch báo cáo	Epoch cuối cùng; không chọn checkpoint theo validation.

5.4 Quét learning rate

Learning rate đỉnh được chọn bằng một lượt quét nhỏ chạy trước các lần chạy chính (Bảng 5.4). Mỗi mức LR chạy 3 epoch trên 20% tập train (140 ảnh mỗi lớp, cân bằng lớp, cố định bằng seed 42), seed 0, và đánh giá trên toàn bộ validation (5 050 ảnh).

Bảng 5.4: Lưới learning rate đỉnh cho mỗi mô hình.

Nhóm chế độ	Lưới LR	Áp dụng cho
Freeze toàn bộ backbone	$\{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$	Linear probing
Fine-tune (quét bằng full fine-tuning)	$\{3 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}\}$	Freeze một phần và full fine-tuning dùng chung một LR
From-scratch	$\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}\}$	From-scratch

- **Tiêu chí chọn:** top-1 trên validation ở *epoch cuối (epoch 3)* của lượt quét; nếu hòa, chọn NLL validation thấp hơn.
- **Quy tắc biên:** nếu LR tốt nhất nằm ở biên lưới, mở rộng thêm một mức (nhân hoặc chia 3) về phía đó và chạy lại riêng mức đó.
- **Hạn chế:** lượt quét ngắn (3 epoch, 20% dữ liệu) có thể thiên về LR lớn hơn mức tối ưu của lần chạy dài, nhất là với from-scratch; kết quả được xem là lựa chọn thực dụng, không phải tối ưu hóa toàn cục.

5.5 Các thí nghiệm và ngân sách tính toán

Số epoch được cố định từ trước vì lịch cosine phụ thuộc tổng số epoch. Bảng 5.5 liệt kê các thí nghiệm theo thứ tự ưu tiên cùng chi phí ước tính (GPU-giờ Tesla T4 trên Colab miễn phí, từ tốc độ đo end-to-end ở Phụ lục C). Phần cam kết khoảng 24,6 GPU-giờ (dự trù thêm khoảng 25% cho khởi động lại phiên, tức khoảng 30 giờ thực tế); các mức mở rộng chỉ chạy khi còn hạn mức Colab, theo đúng thứ tự trong bảng, và mỗi mức độc lập nên việc dừng sớm vẫn cho báo cáo hoàn chỉnh.

Bảng 5.5: Thí nghiệm theo thứ tự ưu tiên và chi phí ước tính.

Mức	Thí nghiệm	Trả lời câu hỏi	GPU-giờ
Cam kết	Quét LR (chạy đầu tiên)	Công bằng giữa kiến trúc	0,8
Cam kết	8 cấu hình lõi, 1 seed (6 pretrained × 10 epoch + 2 scratch × 40 epoch)	RQ1, RQ2, RQ3	9,6
Cam kết	Ablation theo kích thước dữ liệu: 10%, 25%, 50% (cân bằng lớp, lồng nhau) × {scratch 40 epoch, full fine-tune 10 epoch} × 2 mô hình	RQ2	6,4
Cam kết	Thêm 2 seed cho 6 cấu hình pretrained (tổng 3 seed)	Phương sai, độ tin cậy	7,8
Mở rộng 1	ConvNeXt-T vs Swin-T, 3 chế độ pretrained	Họ kiến trúc hay đời thiết kế	4,7
Mở rộng 2	DeiT-S from-scratch 100 epoch (đối chứng bất đối xứng)	Khoảng cách của ViT có do huấn luyện quá ngắn không	7,3
Mở rộng 3	ConvNeXt-T và Swin-T from-scratch 40 epoch	Lặp lại so sánh scratch với cặp 2	8,4

5.6 Giao thức đánh giá và phân tích

Chỉ số. Top-1, top-5 và macro-F1 trên tập test; số tham số trainable và GMACs; thời gian mỗi epoch và tổng thời gian huấn luyện; thông lượng và độ trễ suy luận; bộ nhớ GPU đỉnh. Cách đo chi phí được định nghĩa ở Mục 5.7.

Phân tích bổ sung (chỉ cần suy luận). Sau các thí nghiệm lõi, mỗi lần chạy đã lưu trọng số cuối, logits validation/test và lịch sử huấn luyện, nên các phân tích sau không cần huấn luyện thêm: (i) ma trận nhầm lẫn và phân tích lỗi, kèm các ảnh huấn luyện có loss cao nhất để minh họa nhiễu nhãn; (ii) hiệu chỉnh độ tin cậy [6]: ECE, NLL, Brier, biểu đồ reliability và temperature scaling fit trên validation; (iii) ensemble CNN+ViT so với mô hình đơn và so với ensemble cùng kiến trúc khác seed; (iv) độ bền vững trước nhiễu Gauss, làm mờ, nén JPEG, đổi tương phản, che khuất và xáo trộn patch [8] trên tập con test 50 ảnh mỗi lớp (sinh bằng seed cố định, không phân phối lại); (v) diễn giải định tính bằng Grad-CAM [12] cho CNN và bản đồ chú ý hoặc Grad-CAM trên token cho ViT trên khoảng 12 ảnh chọn trước, dùng như minh họa chứ không phải bằng chứng định lượng.

Về mất cân bằng lớp. Food-101 cân bằng tuyệt đối (700/50/250 ảnh mỗi lớp trong split của nhóm), nên không dùng reweighting, oversampling hay focal loss; vấn đề thực tế là nhiễu nhãn ở tập train.

5.7 Cách đo chi phí

Mọi số liệu chi phí được đo bằng cùng một script, trên cùng một GPU trong cùng một phiên, và báo cáo ghi rõ loại GPU.

- **Thời gian mỗi epoch, chỉ GPU:** 70 700 ảnh chia cho tốc độ huấn luyện đo được (img/s) với batch 64, AMP fp16, AdamW, batch tổng hợp đặt sẵn trên GPU (không có bộ nạp dữ liệu). Đây là cột dùng để so sánh các kiến trúc và chế độ freeze.
- **Thời gian mỗi epoch, đo thực trên Colab:** thời gian thực mỗi epoch (huấn luyện cộng validation) trong lần huấn luyện thật. Cột này bị giới hạn bởi bộ nạp dữ liệu trên 2 nhân CPU nên các chế độ freeze gần như không nhanh hơn (Phụ lục C).
- **Tổng thời gian huấn luyện:** tổng thời gian các epoch, không tính thời gian chờ giữa các phiên hoặc khởi động lại.
- **Bộ nhớ GPU đỉnh:** `torch.cuda.max_memory_allocated` (trọng số, activation, gradient, trạng thái optimizer khi huấn luyện; trọng số và activation khi suy luận).
- **Suy luận:** AMP fp16, `torch.inference_mode`, batch 1 và 128, đồng bộ CUDA quanh mỗi lần gọi; độ trễ là median (và p95) của một lần gọi, thông lượng bằng batch chia độ trễ median.

Chương 6

KẾT QUẢ THỰC NGHIỆM

Chương này sẽ được hoàn thiện ở mốc A1.2. Cấu trúc bảng và cách đo đã được cố định ở Chương 5; mọi con số được sinh tự động từ các tệp `results/*.json`*, không nhập tay.*

6.1 Độ chính xác và F1

Bảng 6.1: Độ chính xác trên tập test (25 250 ảnh); mean ± std trên 3 seed với cấu hình pretrained, 1 seed với from-scratch.

Họ	Kiến trúc	Chế độ	Trainable	Top-1 (%)	Top-5 (%)	Macro-F1 (%)
CNN	ResNet-50	From-scratch	100%	*[. . .]*	*[. . .]*	*[. . .]*
CNN	ResNet-50	Linear probing	0,9%	*[. . .]*	*[. . .]*	*[. . .]*
CNN	ResNet-50	Freeze một phần (layer4 + đầu ra)	64,0%	*[. . .]*	*[. . .]*	*[. . .]*
CNN	ResNet-50	Full fine-tuning	100%	*[. . .]*	*[. . .]*	*[. . .]*
Transformer	DeiT-S	From-scratch	100%	*[. . .]*	*[. . .]*	*[. . .]*
Transformer	DeiT-S	Linear probing	0,2%	*[. . .]*	*[. . .]*	*[. . .]*
Transformer	DeiT-S	Freeze một phần (block 7–12 + đầu ra)	49,2%	*[. . .]*	*[. . .]*	*[. . .]*
Transformer	DeiT-S	Full fine-tuning	100%	*[. . .]*	*[. . .]*	*[. . .]*

6.2 Diễn biến quá trình huấn luyện

[sẽ cập nhật] – đường cong validation theo epoch của các chế độ huấn luyện

Hình 6.1: So sánh độ chính xác và tốc độ thích ứng giữa các chiến lược huấn luyện.

6.3 Ma trận nhầm lẫn

[sẽ cập nhật] – ma trận nhầm lẫn 101×101 (rút gọn thành các cặp lớp bị nhầm nhiều nhất) cho từng mô hình ở full fine-tuning.

Bảng 6.2: Các cặp lớp bị nhầm nhiều nhất trên test (full fine-tuning).

ResNet-50	DeiT-S	Chung cho cả hai
[. . .]	*[. . .]*	*[. . .]*

6.4 Chi phí tính toán

6.4.1 Chi phí huấn luyện

Bảng 6.3: Chi phí huấn luyện. Cột "chỉ GPU" dùng để so sánh; cột "Colab" là thời gian thực trong lần huấn luyện thật và bị giới hạn bởi bộ nạp dữ liệu (xem Mục 5.7).

Kiến trúc	Chế độ	Trainable (M / %)	Epoch	s/epoch (chỉ GPU)	phút/epoch (Colab)	Tổng (giờ)	Peak mem (MB)
ResNet-50	From-scratch	23,71 / 100%	40	205,0	*[. . .]*	*[. . .]*	3206
ResNet-50	Linear probing	0,21 / 0,9%	10	70,8	*[. . .]*	*[. . .]*	517
ResNet-50	Freeze một phần	15,17 / 64,0%	10	87,6	*[. . .]*	*[. . .]*	692
ResNet-50	Full fine-tuning	23,71 / 100%	10	205,0	*[. . .]*	*[. . .]*	3206
DeiT-S	From-scratch	21,70 / 100%	40	197,3	*[. . .]*	*[. . .]*	2550
DeiT-S	Linear probing	0,04 / 0,2%	10	60,0	*[. . .]*	*[. . .]*	333
DeiT-S	Freeze một phần	10,69 / 49,2%	10	126,1	*[. . .]*	*[. . .]*	1383
DeiT-S	Full fine-tuning	21,70 / 100%	10	197,3	*[. . .]*	*[. . .]*	2550

6.4.2 Chi phí suy luận

Bảng 6.4: Chi phí suy luận (chỉ phụ thuộc kiến trúc; AMP fp16; Tesla T4, PyTorch 2.11.0+cu130, timm 1.0.30, đo bằng `cost_bench.py`). Latency batch 1 bị chi phối bởi chi phí khởi chạy kernel nên không tỷ lệ với GMACs. Hai lần đo trên hai phiên lệch nhau tới khoảng 7%; đây là lần đo thứ hai.

Kiến trúc	Tham số (M)	GMACs	img/s (batch 128)	Latency batch 1 (ms): median / p95	Peak mem (MB): batch 1 / 128
ResNet-50	23,71	4,087	950,7	7,21 / 8,63	170 / 871
DeiT-S	21,70	4,599	1102,8	14,47 / 19,64	152 / 518
ConvNeXt-T (mở rộng)	27,90	4,455	658,7	14,08 / 18,26	183 / 1278
Swin-T (mở rộng)	27,60	4,490	444,9	23,59 / 29,03	187 / 1766

6.5 Ablation theo kích thước dữ liệu

[sẽ cập nhật] – đồ thị độ chính xác theo tỷ lệ dữ liệu huấn luyện (10%, 25%, 50%, 100%) cho from-scratch và full fine-tuning của hai mô hình.

6.6 Kết quả phân tích bổ sung (ngoài yêu cầu tối thiểu)

- **Phân tích lỗi:** hiệu năng theo lớp, ca sai của CNN so với ViT, ảnh huấn luyện có loss cao nhất. *[sẽ cập nhật]*
- **Hiệu chỉnh:** ECE, NLL, Brier, biểu đồ reliability, trước và sau temperature scaling. *[sẽ cập nhật]*
- **Ensemble:** CNN+ViT so với mô hình đơn và so với ensemble cùng kiến trúc; mức chồng lấn lỗi. *[sẽ cập nhật]*
- **Độ bền vững:** độ giảm độ chính xác tương đối theo loại và mức nhiễu. *[sẽ cập nhật]*
- **Diễn giải:** Grad-CAM và bản đồ chú ý trên các ảnh chọn trước. *[sẽ cập nhật]*

Chương 7

PHÂN TÍCH VÀ THẢO LUẬN

Chương này sẽ được viết ở mốc A1.2 dựa trên số liệu thực; dưới đây là khung lập luận và các bằng chứng sẽ dùng cho từng mục. Mọi nhận định phải được đối chiếu với phương sai giữa các seed và với các hạn chế ở Mục 9.2.

7.1 CNN và Transformer

So sánh ResNet-50 với DeiT-S ở từng chế độ (Bảng 6.1), và ConvNeXt-T với Swin-T nếu được chạy, để xem khác biệt có nhất quán giữa hai cặp (do họ kiến trúc) hay chỉ xuất hiện ở một cặp (do đời thiết kế hoặc công thức pretrain). *[sẽ cập nhật]*

7.2 Vai trò của pretraining

Khoảng cách giữa from-scratch và pretrained của từng họ tại 10, 25, 50 và 100% dữ liệu (Mục 6.5); đối chứng 100 epoch cho DeiT-S nếu được chạy, để tách "ViT cần pretrain" khỏi "huấn luyện 40 epoch chưa đủ". Cần nhắc rằng lợi thế của pretrain có thể một phần đến từ sự gần gũi miền dữ liệu với ImageNet-1k (Mục 9.2). *[sẽ cập nhật]* (RQ2)

7.3 Ảnh hưởng của fine-tuning và freeze

Đối chiếu độ chính xác của ba chế độ freeze với chi phí (thời gian chỉ GPU, bộ nhớ đỉnh; Bảng 4.1) và với ablation theo kích thước dữ liệu để trả lời khi nào freeze có lợi. *[sẽ cập nhật]* (RQ1)

7.4 Phân tích lỗi

7.4.1 Hiệu năng theo lớp

Các lớp dễ và khó nhất của từng mô hình, và mức tương quan giữa hai mô hình. *[sẽ cập nhật]*

7.4.2 Các mẫu dự đoán sai

Ví dụ ảnh bị phân loại sai, tách giữa lỗi do nhiễu nhãn (kiểm tra bằng các ảnh huấn luyện có loss cao nhất) và lỗi do hai món thực sự giống nhau. *[sẽ cập nhật]*

7.4.3 Kiểu lỗi của CNN và Transformer

Tỷ lệ lỗi chồng lấn giữa hai họ; những loại ảnh mà một họ sai còn họ kia đúng. Mức chồng lấn này cũng là cơ sở để đánh giá lợi ích của ensemble. *[sẽ cập nhật]* (RQ4)

7.5 Hiệu chỉnh, ensemble, độ bền vững và diễn giải

Đây là phần mở rộng ngoài yêu cầu tối thiểu. Hiệu chỉnh độ tin cậy trước và sau temperature scaling; ensemble CNN+ViT so với mô hình đơn và ensemble cùng kiến trúc; độ giảm độ chính xác trước từng loại nhiễu; bản đồ Grad-CAM và chú ý trên các ảnh được chọn trước (định tính, không phải bằng chứng nhân quả). *[sẽ cập nhật]*

7.6 Đánh đổi giữa hiệu năng và chi phí

Biểu đồ độ chính xác theo chi phí (thời gian huấn luyện chỉ GPU, GMACs, độ trễ và thông lượng suy luận, bộ nhớ đỉnh) cho từng kiến trúc và chế độ, để xác định cấu hình nằm trên đường Pareto (Bảng 6.3, 6.4). Lưu ý rằng trên Colab, thời gian thực của các chế độ freeze gần như bằng nhau vì bộ nạp dữ liệu là nút cổ chai, trong khi số đo chỉ GPU cho thấy khác biệt thật của kiến trúc, và độ trễ batch 1 không tỷ lệ với GMACs. *[sẽ cập nhật]* (RQ3)

Chương 8

TÁI LẬP THÍ NGHIỆM

Nhóm tách bạch **quy trình tái lập chuẩn** (chỉ cần mã nguồn và báo cáo) với **cách nhóm thực sự chạy** (các tối ưu để tiết kiệm thời gian). Nguyên tắc: tối ưu chỉ thay đổi *kết quả của một bước đến từ đâu*, không thay đổi *bước đó tính gì*.

Bảng 8.1: Quy trình chuẩn và cách nhóm thực sự chạy.

Bước	Quy trình chuẩn (chỉ repo)	Cách nhóm thực sự chạy
Dữ liệu	Tải Food-101 bằng torchvision, resize cạnh ngắn 256 bằng script của repo, chia theo `split.json` đã commit (kiểm tra SHA-256 và số ảnh khi khởi động)	Khôi phục bản đã resize từ một tệp tar trên Drive riêng của nhóm (không phân phối công khai); cùng mã, cùng kết quả
Huấn luyện	Mỗi lần chạy một tệp cấu hình YAML đã commit, chạy bằng lệnh CLI; có chế độ smoke test nhỏ (dưới 10 phút trên một GPU bất kỳ)	Hàng đợi chạy trên Colab, lưu checkpoint nguyên tử lên Drive dùng chung, tự động tiếp tục khi mất phiên (đã kiểm tra khôi phục RNG cho kết quả trùng khớp từng bit trên mô hình thử)
Trọng số pretrained	timm tải theo tên và tag cố định	Cache tải sẵn, dùng chung
Kết quả	`results/*.json` (số liệu, đường cong, cấu hình, môi trường) trong repo; bảng và hình sinh lại từ JSON bằng notebook; trọng số cuối và logits đăng trên GitHub Release kèm checksum, có lệnh đánh giá lại để tái hiện số liệu test trong vài phút	Checkpoint và log trung gian nằm trên Drive (không phải sản phẩm nộp)

- **Môi trường:** đã kiểm tra với Python 3.13, PyTorch 2.11.0, torchvision 0.26.0, timm 1.0.30 trên Tesla T4. Tệp yêu cầu được tách theo bài (`requirements/base.txt`, `requirements`

`/a1.txt`); `a1.txt` ghim `timm==1.0.30`, còn PyTorch và torchvision không được ghim vì Colab dùng bản dựng CUDA có sẵn (cài đè sẽ tải lại hàng GB và có thể làm hỏng runtime); các phiên bản đã kiểm tra được ghi trong README. Cấu hình khoa học (mô hình, chế độ, epoch, LR, seed, augmentation) nằm trong YAML được commit; đường dẫn và tham số máy (thư mục dữ liệu, số worker) đi qua biến môi trường hoặc tham số CLI.

- **Chi phí tái lập đầy đủ:** khoảng 25 GPU-giờ T4. Sai khác giữa các lần chạy được ước lượng từ 3 seed và nêu trong báo cáo; kết quả không bảo đảm trùng từng bit giữa các loại GPU khác nhau.
- **Cấu trúc mã:** mã của A1 nằm trong thư mục `A1/`; các tiện ích dùng chung giữa các bài (seed và ghi môi trường, lưu checkpoint nguyên tử, ghi kết quả JSON, đo chi phí) nằm trong package `common/` cài được từ thư mục gốc repo (`pipinstall-e.`). Thí nghiệm là notebook mỏng gọi vào package và các script `python-m`.
- **Kiểm tra "fresh clone"** trước khi nộp A1.2: clone repo trên một runtime Colab mới, không có Drive hay dữ liệu cache, rồi chỉ làm theo README để chạy smoke test và đánh giá lại các trọng số đã phát hành.
- **Phát hành:** trọng số cuối và logits đăng trên GitHub Release kèm `SHA256SUMS`. Nhóm không đăng dữ liệu gốc, bản cache đã resize, hay ảnh đã làm nhiễu.

Chương 9

KẾT LUẬN

9.1 Tổng kết kết quả

Sẽ hoàn thiện ở mốc A1.2. Phần này tóm tắt câu trả lời cho RQ1–RQ4 dựa trên số liệu thực, mỗi câu trả lời kèm con số cụ thể và mức tin cậy (phương sai giữa các seed). *[sẽ cập nhật]*

9.2 Hạn chế và hướng phát triển

- **Chưa kiểm chứng trùng lặp với dữ liệu pretrain.** Nhóm không kiểm tra độc lập sự trùng lặp ở cấp ảnh giữa Food-101 và ImageNet-1k, nên lợi thế của pretrain có thể một phần phản ánh sự gần gũi về miền dữ liệu, chứ không hoàn toàn là khả năng chuyển giao sang dữ liệu chưa từng thấy.
- **Công thức pretrain khác nhau** giữa trọng số CNN và Transformer (Chương 3), nên khác biệt giữa hai họ không hoàn toàn quy được cho kiến trúc.
- **From-scratch 40 epoch** có thể chưa đủ để ViT hội tụ; đối chứng 100 epoch chỉ chạy nếu còn tài nguyên.
- **Công thức augmentation cố định, không tinh chỉnh.** RandAugment nhẹ (30% biên độ) kết hợp Mixup/CutMix áp dụng cho mọi batch có thể là điều chuẩn hóa quá mạnh cho lịch huấn luyện ngắn, nhất là fine-tune 10 epoch hoặc huấn luyện từ đầu 40 epoch.
- **Quét learning rate ngắn** (3 epoch, 20% dữ liệu) và chỉ một tập dữ liệu; nhiễu nhãn ở tập huấn luyện giới hạn độ chính xác tối đa và validation (lấy từ phần train nhiễu) chỉ dùng chọn siêu tham số.
- **Phần cứng Colab không đồng nhất** giữa các phiên (loại GPU, 2 nhân CPU); vì vậy số liệu chi phí được đo kiểm soát trên một GPU, và độ chính xác không bảo đảm trùng từng

bit giữa các lần chạy.

- **Phân tích diễn giải** bằng bản đồ nhiệt là định tính và không phải bằng chứng nhân quả.

Hướng phát triển. Kiểm chứng mức trùng lặp ở cấp ảnh với ImageNet-1k; tinh chỉnh công thức augmentation; lịch huấn luyện dài hơn cho ViT; mở rộng sang tập dữ liệu thứ hai.

TÀI LIỆU THAM KHẢO

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, 2014. `https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/`.

[2] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020. arXiv:1909.13719.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.11929.

[4] ETH Zürich Computer Vision Lab. ethz/food101 (dataset card, licensing information). Hugging Face Datasets. Truy cập 21/10/2026.

[5] Foodspotting. Terms of service (bản lưu trữ 23/12/2017; hiệu lực từ 17/10/2012). Internet Archive.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[8] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1903.12261.

[9] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. arXiv:2103.14030.

[10] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2201.03545.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[12] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. arXiv:1610.02391.

[13] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. arXiv:1512.00567.

[14] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2012.12877.

[15] Ross Wightman. PyTorch image models (timm), 2019. `https://github.com/huggingface/pytorch-image-models`.

[16] Ross Wightman, Hugo Touvron, and Hervé Jégou. ResNet strikes back: An improved training procedure in timm. *arXiv preprint arXiv:2110.00476*, 2021.

[17] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1905.04899.

[18] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.09412.

Phụ lục A

Ví dụ tệp cấu hình một lần chạy

Mỗi lần chạy có một tệp YAML được commit; chỉ các giá trị ảnh hưởng đến kết quả khoa học nằm trong tệp này. Đường dẫn dữ liệu và checkpoint đi qua biến môi trường. Giá trị `lr_peak` được điền từ kết quả quét LR (Mục 5.4).

```
run_id: a1_resnet50_full_s0
model: resnet50.a1_in1k
pretrained: true
mode: full                # scratch | head | partial | full
epochs: 10
seed: 0
data_fraction: 1.0        # 0.1 | 0.25 | 0.5 | 1.0 (ablation)
batch_size: 64
lr_peak: TBD              # from the LR sweep (validation)
warmup_frac: 0.05
min_lr_frac: 0.01
weight_decay: 0.05
drop_rate: 0.0
drop_path_rate: 0.0
amp: fp16
augmentation:
  random_resized_crop:
    size: 224
    scale: [0.25, 1.0]
    ratio: [0.75, 1.3333]
  hflip_p: 0.5
  randaugment:
    num_ops: 2
    magnitude: 9
    num_magnitude_bins: 31
  mixup:
    mixup_alpha: 0.8
    cutmix_alpha: 1.0
```

```
    prob: 1.0
    switch_prob: 0.5
    label_smoothing: 0.1
split_json_sha256_16: 75ed81bfe380c089
```

Phụ lục B

Kết quả thực nghiệm bổ sung

Sẽ được hoàn thiện ở mốc A1.2.

- Kết quả đầy đủ của lượt quét learning rate: top-1 và NLL validation của từng mức LR, từng mô hình và nhóm chế độ, và mức LR được chọn (Mục 5.4). *[sẽ cập nhật]*
- Kết quả từng seed của các cấu hình pretrained (trước khi lấy trung bình). *[sẽ cập nhật]*
- Chi tiết độ bền vững: độ chính xác theo từng loại và mức nhiễu. *[sẽ cập nhật]*
- Biểu đồ reliability trước và sau temperature scaling. *[sẽ cập nhật]*
- Các ví dụ Grad-CAM và bản đồ chú ý bổ sung. *[sẽ cập nhật]*

Phụ lục C

Đo sơ bộ trên Colab (Tesla T4)

Các số liệu dưới đây là kết quả đo sơ bộ dùng để lập ngân sách tính toán ở Mục 5.5 (đo bằng các script thăm dò, không phải `cost_bench.py`); số liệu chi phí dùng trong so sánh nằm ở Mục 6.4, đo bằng một script duy nhất theo quy ước ở Mục 5.7, nên có thể lệch nhẹ so với các bảng dưới đây. Môi trường: Google Colab miễn phí, Tesla T4 (15 GB), 2 nhân CPU, RAM 12,7 GB; Python 3.13.15, PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130, timm 1.0.30. Batch 64, AMP fp16, ảnh 224×224.

Bảng C.1: Thông lượng bộ nạp dữ liệu (img/s) trên ảnh đã resize 256 px, batch 64. "Float" chuẩn hóa trên CPU; "uint8" chuẩn hóa trên GPU.

Biến thể	2 worker	3 worker
Float, RandAugment (nền tham chiếu)	278,7	268,7
uint8, RandAugment (được chọn)	371,9	344,7
uint8, TrivialAugmentWide	413,8	423,4
uint8, không augmentation nặng	542,4	462,1

Bộ nạp validation (Resize + CenterCrop, uint8) đạt 1 068 img/s. Với 2 nhân CPU, hai worker là tốt nhất cho cấu hình được chọn.

Bảng C.2: Tốc độ huấn luyện chỉ GPU (batch tổng hợp trên GPU) và bộ nhớ GPU đỉnh.

	img/s			Peak mem (MB)		
Kiến trúc	Head	Partial	Full	Head	Partial	Full
ResNet-50	983,8	788,8	343,4	517	692	3 206
DeiT-S	1 158,5	547,9	350,2	333	1 383	2 550
ConvNeXt-T	661,3	542,6	223,9	728	908	4 400
Swin-T	439,6	381,2	168,1	968	1 147	4 879

Bảng C.3: Tốc độ huấn luyện end-to-end (bộ nạp uint8 + RandAugment, 2 worker, Mixup/CutMix) và tỷ lệ thời gian chờ dữ liệu.

	img/s			Chờ dữ liệu (%)		
Kiến trúc	Head	Partial	Full	Head	Partial	Full
ResNet-50	338,0	336,6	281,7	66	59	14
DeiT-S	334,3	309,6	275,2	69	40	14
ConvNeXt-T	337,0	319,9	223,9	52	45	0
Swin-T	308,4	304,3	170,9	34	27	0

Nhận xét. Ở chế độ head-only và freeze một phần, huấn luyện chờ dữ liệu 27–69% thời gian nên thời gian thực mỗi epoch gần như không đổi giữa các chế độ freeze trên Colab (khoảng 3,6–4,0 phút mỗi epoch gồm cả validation), dù tốc độ chỉ GPU chênh nhau gấp nhiều lần. Đây là lý do cột "chỉ GPU" được dùng để so sánh chi phí giữa các kiến trúc và chế độ. Với full fine-tuning, ResNet-50 và DeiT-S gần như bị giới hạn bởi GPU còn ConvNeXt-T và Swin-T hoàn toàn bị giới hạn bởi GPU.